

SentinelAI

AI Usage & Development Report

AI-First Startup Hackathon | HackIndia 2026

Project	SentinelAI
Team	Team SentinelAI
Participant	Varun Sharma
Track	AI SaaS Startup
Live Link	https://sentinelai-defence.netlify.app
Repository	HackIndiaXYZ / ai-first-startup-hackathon-build-a-startup-using-ai-only-sentinelai

1. Executive Summary

SentinelAI is an AI-powered security intelligence platform designed to help developers understand, prioritize, and remediate application-security findings. It acts as an AI security engineer that analyzes authorized security signals, explains risks in plain language, prioritizes important findings, provides remediation guidance, and supports persistent conversations.

SentinelAI was developed using an AI-first workflow in which AI was the primary builder across ideation, architecture, UI/UX, implementation, debugging, testing, security review, documentation, deployment, and presentation preparation. Human involvement focused on requirements, product decisions, validation, testing, and refinement.

2. AI-First Development Principle

The HackIndia AI-First Startup Hackathon requires AI to be the primary builder, minimal manual-first development, documentation of AI usage, and an AI-driven execution workflow. SentinelAI was built around this principle rather than using AI only as a conventional coding assistant.

- AI was used across the product lifecycle, not only for isolated code snippets.
- AI generated and refined architecture, UI/UX, application logic, APIs, database workflows, debugging approaches, tests, documentation, and presentation material.
- Human decisions determined requirements, product direction, acceptance criteria, and final validation.
- Generated work was executed, reviewed, tested, debugged, and iterated before being accepted.

3. AI Tools Used

Tool / Technology	Primary Use	Role
ChatGPT	Planning, architecture, coding, debugging, testing, security review, documentation, pitch preparation	Primary AI builder / reasoning partner
Claude	Code generation, UI/UX iteration, debugging, architecture refinement	AI coding and product-development agent
GitHub	Version control and submission	Source and delivery infrastructure
Next.js / React	Web application	Product runtime
Prisma + PostgreSQL / Neon	Persistence and database workflows	Data layer
Groq	AI inference	Runtime AI engine

Aikido Security	Application security layer	Security monitoring / firewall
Netlify	Production deployment	Hosting

4. AI-First Development Workflow

4.1 Ideation

AI helped transform the security problem into the SentinelAI concept, positioning, feature set, and user workflow.

4.2 Architecture

AI was used to reason through Next.js structure, API boundaries, authentication, database persistence, AI pipeline, and deployment architecture.

4.3 UI/UX

AI generated and refined the dark security-intelligence interface, dashboard structure, navigation, hierarchy, and interaction patterns.

4.4 Implementation

AI generated substantial application code including pages, components, API routes, authentication logic, database interactions, validation, and user flows.

4.5 Debugging

AI assisted with build failures, runtime errors, hydration issues, database/session problems, and deployment problems.

4.6 Testing

AI helped define test scenarios and edge cases across authentication, projects, findings, analysis, reports, conversations, and settings.

4.7 Security Review

AI reviewed API boundaries, authentication, ownership checks, validation, input limits, safe errors, and security-layer integration.

4.8 Deployment

AI assisted with production configuration, build troubleshooting, deployment readiness, and verification.

4.9 Documentation & Presentation

AI was used to structure project documentation, the AI usage report, demo narrative, and pitch content.

5. Representative Prompt Categories

Development used iterative prompts: define the goal, provide current context, request an implementation or analysis, run the result, report errors, and refine. The examples below represent the prompt categories used during development.

Category	Representative Prompt
Product	Design an AI-first security SaaS that helps developers understand, prioritize, and remediate authorized application security findings.
Architecture	Design a production-ready Next.js architecture for SentinelAI with authentication, projects, findings, reports, AI analysis, conversation persistence, and secure APIs.

UI/UX	Create a dark security-intelligence dashboard with clear navigation, strong hierarchy, and a polished SaaS experience.
AI Analysis	Design an AI analysis flow that converts authorized security signals into structured findings with severity, confidence, explanation, and remediation guidance.
Memory	Make Ask Sentinel behave as one persistent chat window that restores the user's latest conversation after refresh while enforcing server-side ownership.
Debugging	Analyze this build/runtime error, identify the root cause, propose the smallest safe fix, and explain how to verify it.
Security	Review this API for authentication, ownership enforcement, validation, input limits, safe errors, and unintended data exposure.
Deployment	Review the production build/deployment failure and provide a minimal fix that preserves the architecture.
Documentation	Turn the completed implementation into concise technical and AI-usage documentation suitable for a hackathon submission.

6. Major AI-Assisted Product Areas

- Authentication — credentials authentication, password hashing, sessions, protected routes, and user-scoped operations.
- Projects — project creation and owner-scoped project access.
- Security Analysis — AI-powered analysis of authorized security signals with structured findings and remediation guidance.
- Findings — severity/status filtering, project scoping, search limits, and presentation.
- Reports — project-scoped security reporting and summaries.
- Ask Sentinel — persistent AI conversations, history retrieval, markdown rendering, and contextual project/finding information.
- Settings — account, password, preference, and account-deletion workflows with validation.
- Security Controls — authentication/authorization boundaries, validation, input limits, safe errors, and Aikido integration.
- Deployment — production build and GitHub-to-Netlify deployment workflow.

7. Case Study: Persistent Sentinel Memory

A major product refinement was making Ask Sentinel persistent. The backend already stored Conversation and Message records, while the interface initially did not restore an existing conversation after refresh. AI-assisted debugging identified the need to retrieve the user's latest conversation and messages during initialization.

- Added authenticated conversation listing and conversation-detail API routes.
- Loaded the latest existing conversation when Ask Sentinel initializes.
- Restored stored messages into the chat interface.
- Preserved the conversation ID for subsequent AI requests.
- Added a 'Restoring Sentinel memory' state to avoid an incorrect empty-state flash.
- Built and tested the application before committing the feature.

This demonstrates the AI-first loop: identify a product gap → reason about the architecture → generate implementation → build → diagnose → validate → commit → deploy.

8. Human Guidance, Validation & Refinement

AI was the primary builder, while the human builder retained product ownership and validation responsibility.

- Choosing the problem, target users, product direction, and feature priorities.
- Providing requirements and deciding which AI-generated approaches to adopt.
- Running builds, development servers, migrations, tests, and production deployments.
- Inspecting generated code and reporting concrete errors for AI-assisted iteration.
- Testing user flows and validating whether behavior matched the intended experience.
- Reviewing security and ownership requirements.
- Making final decisions about scope, architecture, UI, and submission readiness.

9. Evidence of AI-Driven Execution

Milestone	Evidence	Outcome
Persistent conversation memory	Git commit 2ef97f0 — feat: add persistent Sentinel conversation memory	Conversation APIs and restored chat history shipped
Production build	Next.js production build completed successfully	Application compiled successfully
Database persistence	Prisma migrations deployed to PostgreSQL/Neon	Application data persists
Production delivery	GitHub main connected to Netlify	Live deployed application

10. AI-First Contribution Statement

SentinelAI was developed under an AI-first workflow in which AI was used as the primary execution engine across the majority of the product lifecycle. The workflow deliberately minimized manual-first implementation: requirements were expressed to AI, AI generated or transformed substantial implementation work, and the human builder executed, evaluated, validated, and refined the results.

The team does not claim an exact percentage of AI-generated versus manually authored lines because a line-level authorship measurement was not maintained. Instead, this report documents the actual workflow and representative evidence transparently.

11. Technology Stack

Layer	Technology
Frontend	Next.js 16.3.3, React, Tailwind-style utility classes
Backend	Next.js API routes / server-side logic
Authentication	Auth.js / NextAuth
Database	PostgreSQL on Neon
ORM	Prisma 7.10.0
AI Inference	Groq — openai/gpt-oss-20b
Security	Aikido Security firewall integration
Deployment	Netlify
Version Control	Git + GitHub

12. Final Outcome

The result is a live AI SaaS security product rather than a static prototype. SentinelAI combines AI-assisted security analysis, prioritized findings, reporting, authenticated project workflows, and persistent AI conversations into a single developer-focused experience.

The project demonstrates the core idea of AI-first startup building: use AI to compress the distance between product idea and deployable software while keeping human judgment responsible for direction, validation, security decisions, and final quality.

13. Transparency Notes

- This report documents representative prompt categories and the development workflow; it is not a verbatim transcript of every prompt.
- The AI contribution percentage is not claimed as an exact measured number because line-level authorship tracking was not maintained.
- AI-generated work was validated through builds, runtime testing, database checks, and production deployment.
- Secrets, API keys, passwords, and other credentials are intentionally excluded from this report.